

汇编语言程序设计

4.3 循环程序结构

目录

1. 循环程序的结构
2. Loop循环指令
3. 计数控制循环
4. 条件分支控制

4.3 学习目标与达成要求

知识目标

- 理解循环程序的结构
- 掌握Loop指令
- 掌握计数控制与条件控制两类结构

能力目标

- 能用循环完成数组求和、求最大值
- 能把C语言思路迁移到汇编实现
- 能根据任务选择合适的循环控制方式

素养目标

- 形成“先分析流程，再写代码”的意识
- 关注边界值与易错点，培养严谨性
- 体会低级语言中程序控制的本质

应该掌握

- 会判断一个循环由“循环初始—循环体—循环控制”哪三部分组成
- 会解释LOOP为什么等价于“DEC ECX + JNZ label”，并知道ECX=0的陷阱
- 会写出至少1个计数控制循环和1个条件控制循环程序
- 会比较“先判断后循环”与“先循环后判断”两种结构的适用场景

学习基础、重点和难点

学习基础

- 已学习：寄存器、寻址方式、顺序结构、分支结构
- 已具备：阅读简单汇编语句与跟踪寄存器变化的能力
- 易出现：只会模仿代码，不会提炼“循环控制条件”

学习重点

- ECX=0 时 LOOP 的边界理解
- 索引、位移与数据类型对应关系
- 先判断/先循环结构的辨析与选择

学习难点

- ECX=0 时 LOOP 的边界理解
- 索引、位移与数据类型对应关系
- 先判断/先循环结构的辨析与选择

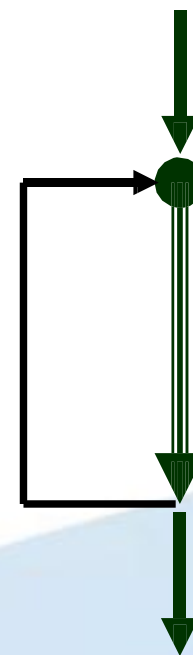
汇编语言程序设计

4.3 循环程序结构

1. 循环程序的结构

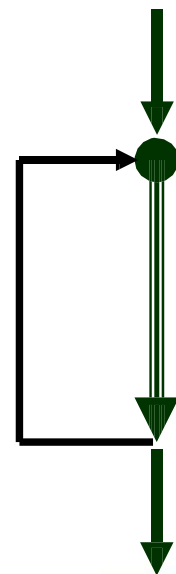
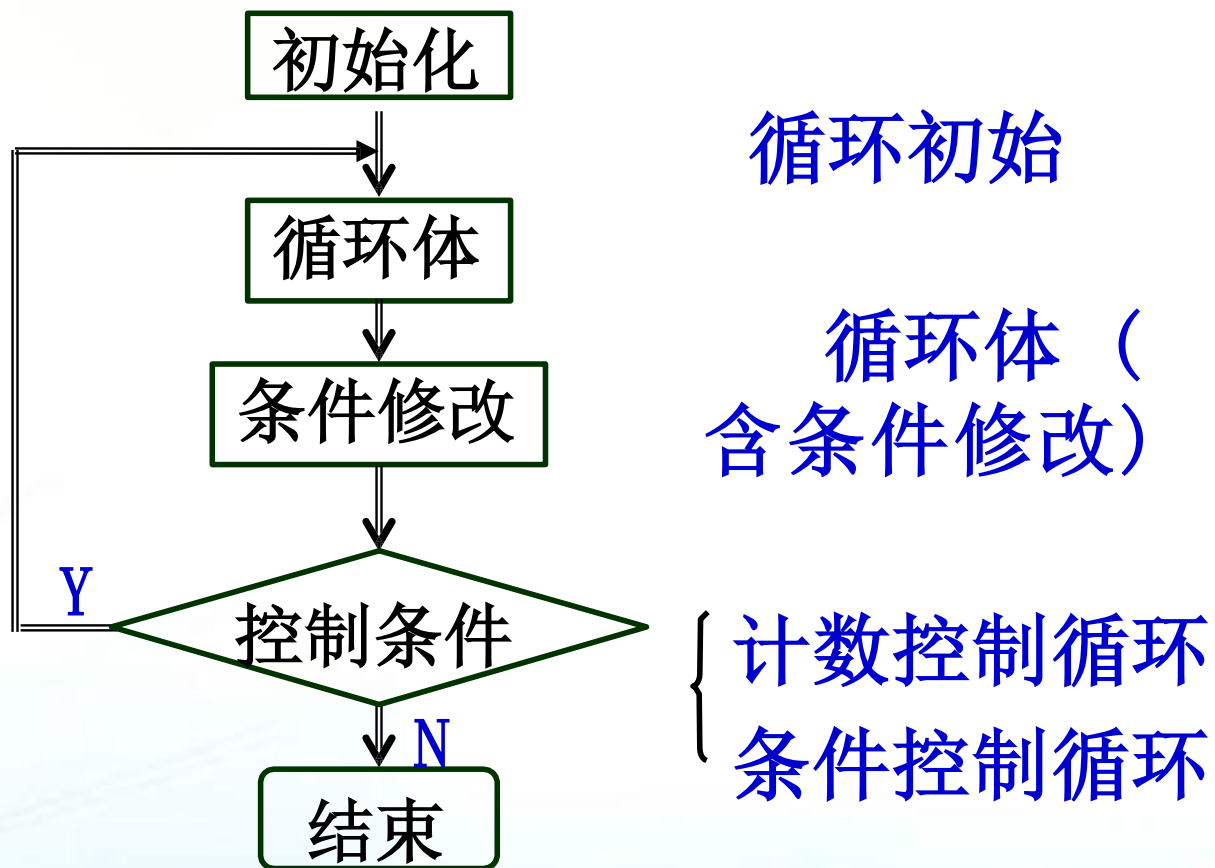
循环程序结构通常有3个组成部分

- 循环初始——为开始循环准备必要的条件
 - 如设置循环次数、循环体需要的初始值等
- 循环体——重复执行的程序代码
 - 其中包括对循环条件的修改等
- 循环控制——判断循环条件是否成立
 - 决定是否继续循环



循环程序流程图

循环程序结构的关键：
如何控制循环



循环案例：数组求和

```
mov ecx,lengthof array
```

```
xor eax,eax
```

```
mov ebx,eax
```

} 循环初始

again:

```
add eax,array[ebx*(type array)]
```

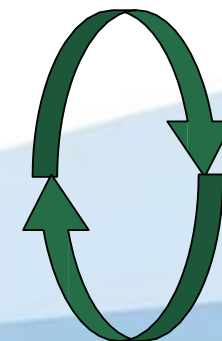
```
inc ebx
```

```
loop again
```

} 循环控制

```
mov sum,eax
```

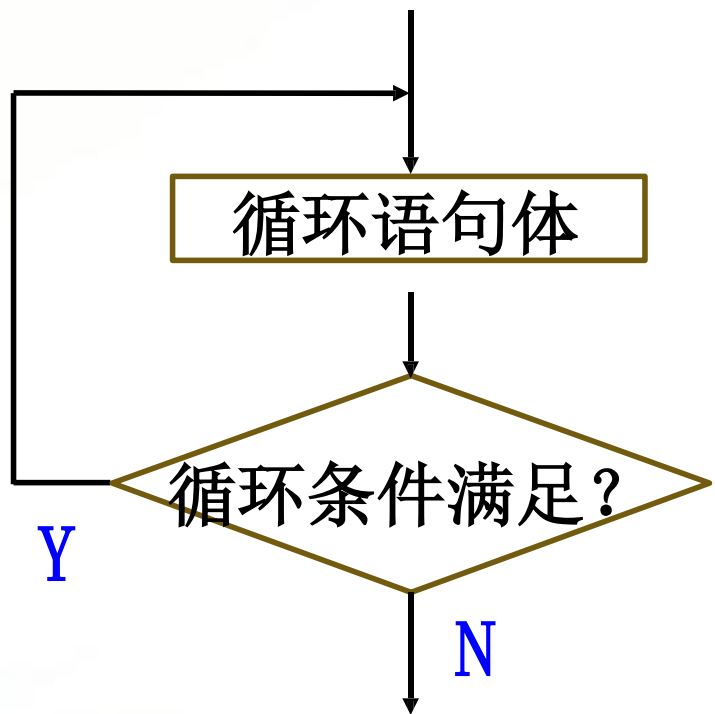
} 循环体



计数控制循环

- 通过次数控制循环
 - 计数可以减量进行，即减到0结束
 - 计数可以增量进行，即达到规定值结束
 - 类似高级语言的for语句
- 利用LOOP指令属于减量计数的循环控制
 - 常见是“先循环、后判断”循环结构
 - 对应高级语言的do语句

“先循环、后判断”结构



LOOP指令的应用

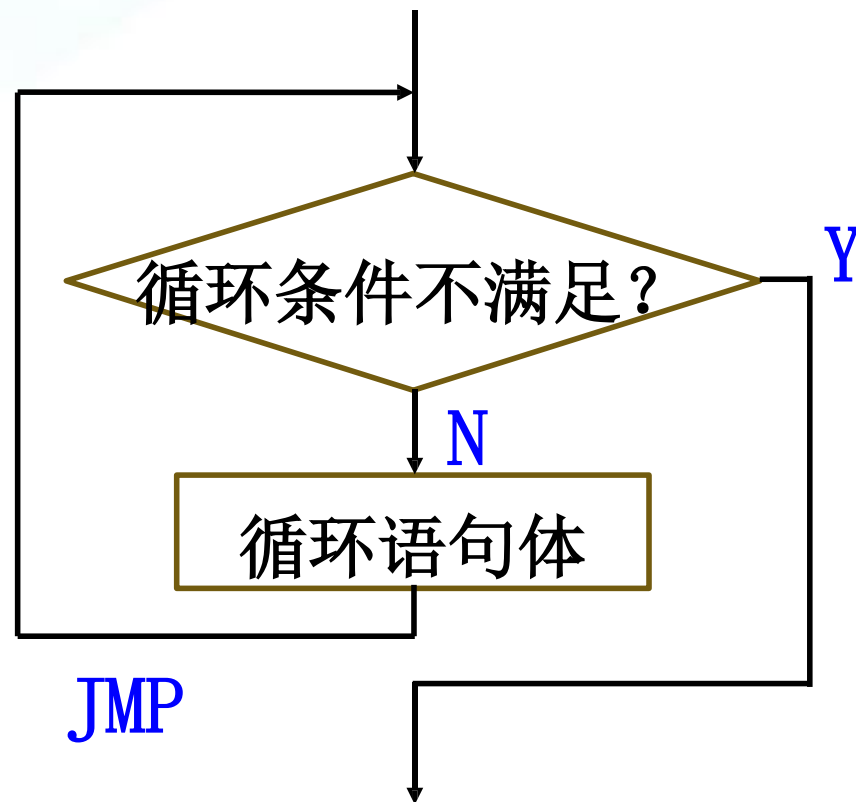
- LOOP 是循环指令，用于实现减量计数的循环控制
- 典型应用形式

```
mov ecx, num           ;设置循环的计数初值num
label:
    loop_code           ;循环体
loop label              ;ECX减1，未到0继续循环
next code              ;到0循环结束，顺序执行
```

条件控制循环

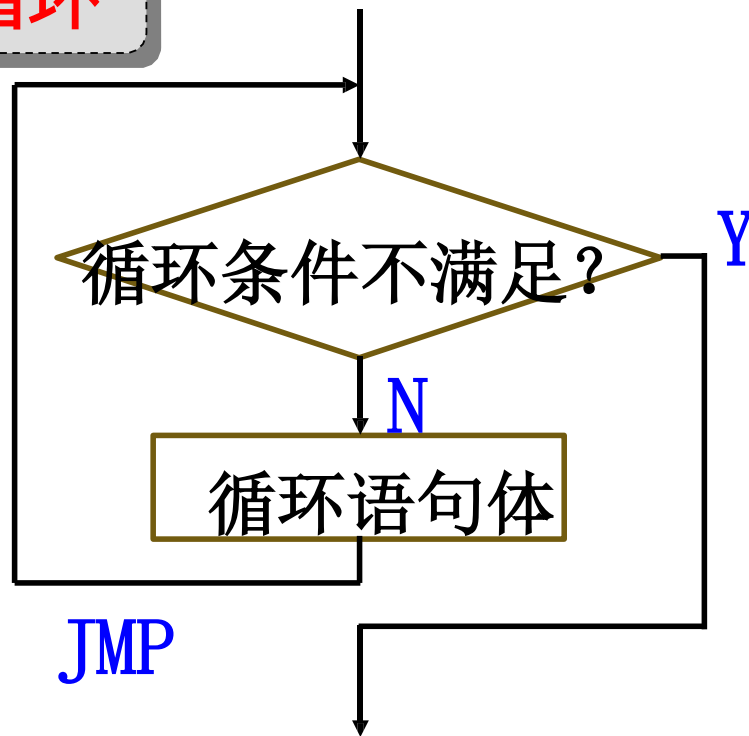
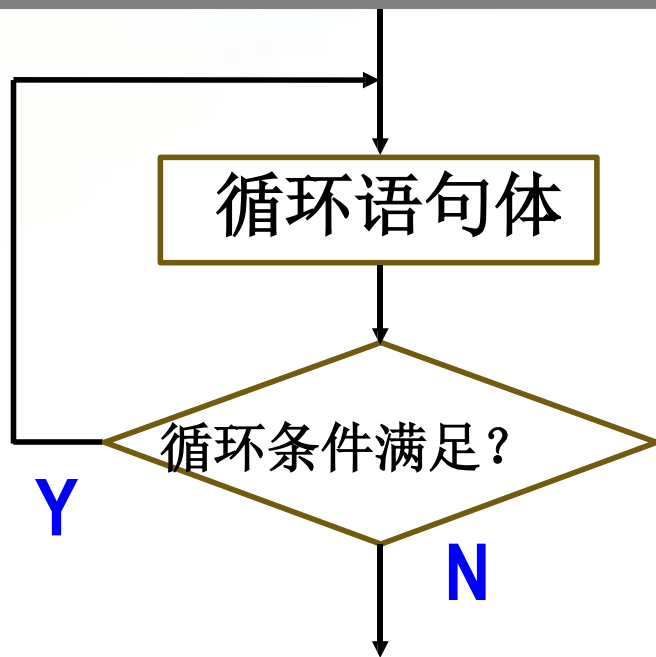
- 根据条件决定是否进行循环
 - 使用比较、测试等指令设置状态标志、产生条件
 - 使用条件转移指令实现循环控制
 - 常需要使用无条件转移指令配合实现循环
- 多见是“先判断、后循环”循环结构
 - 对应高级语言的while语句

“先判断、后循环”结构



两种结构的对比

LOOP指令仅实现了简单的计数循环



条件转移指令才能实现复杂的循环控制

汇编语言程序设计

4.3 循环程序结构

2. LOOP循环指令

4.3 学习目标与达成要求

知识目标

- 掌握Loop指令

能力目标

- 能用循环完成数组求和、求最大值
- 能把C语言思路迁移到汇编实现

素养目标

- 形成“先分析流程，再写代码”的意识
- 关注边界值与易错点，培养严谨性
- 体会低级语言中程序控制的本质

应该掌握

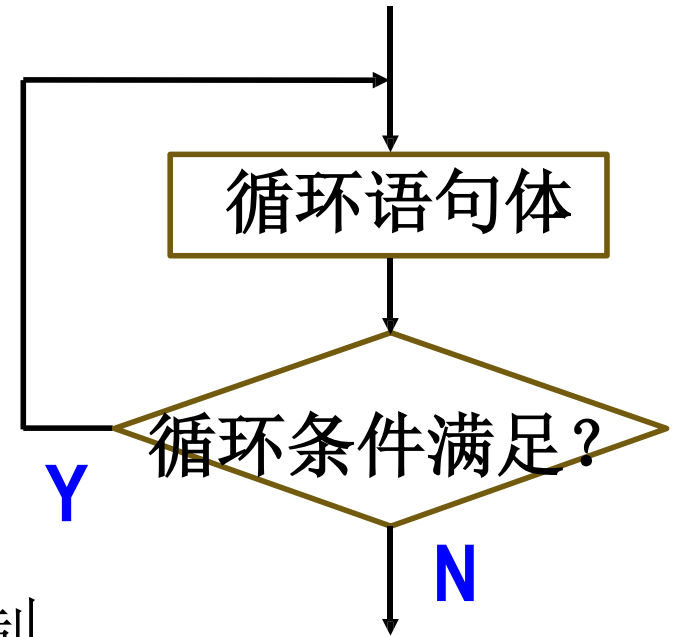
- 会解释LOOP为什么等价于“DEC ECX + JNZ label”，并知道ECX=0的陷阱
- 会写出至少1个计数控制循环

复习：计数控制循环

- 通过次数控制循环
 - 计数可以减量进行，即减到0结束
 - 计数可以增量进行，即达到规定值结束
 - 类似高级语言的for语句
- 利用LOOP指令属于减量计数的循环控制
 - 常见是“先循环、后判断”循环结构
 - 对应高级语言的do语句

LOOP循环

- 通过次数控制循环
 - 计数可以减量进行，即减到0结束
 - 计数可以增量进行，即达到规定值结束
- 常见是“先循环、后判断”循环结构
- LOOP 是循环指令，用于实现减量计数的循环控制



L00P循环指令的作用

L00P label

;功能1: ECX=ECX-1

;功能2: 若ECX≠0, 转移到LABEL

;否则, 顺序执行

相当于

DEC ECX

相当于

JNZ label

DEC ECX

JNZ label

寄存器ECX是默认的计数器 目标
地址采用相对短转移

LOOP指令的应用

- LOOP 是循环指令，用于实现减量计数的循环控制
- 典型应用形式

```
mov ecx, num           ;设置循环的计数初值num
label:
    loop_code           ;循环体
loop label              ;ECX减1，未到0继续循环
next code               ;到0循环结束，顺序执行
```

LOOP指令的循环次数

```
mov ecx, num           ; 设置循环的计数初值num
label:
    loop_code           ; 循环体
loop label               ; ECX减1, 未到0继续循环
next code                ; 到0循环结束, 顺序执行
```

循环初值	1	2		$2^{32}-1$	0
循环次数	1	2		$2^{32}-1$	2^{32}

LOOP指令：先减1后判断

```
mov ecx, 0           ; 设置循环的计数初值0
label:
    loop_code        ; 循环体
loop label            ; ECX减1，未到0继续循环
next code             ; 到0循环结束，顺序执行
```

$ECX = 0 - 1 = -1 \neq 0$ ，继续循环

“-1”的32位补码 = FFFFFFFFH = $2^{32} - 1$

1次

$2^{32} - 1$ 次

$ECX = 0$ ，共计循环 2^{32} 次

JECXZ指令

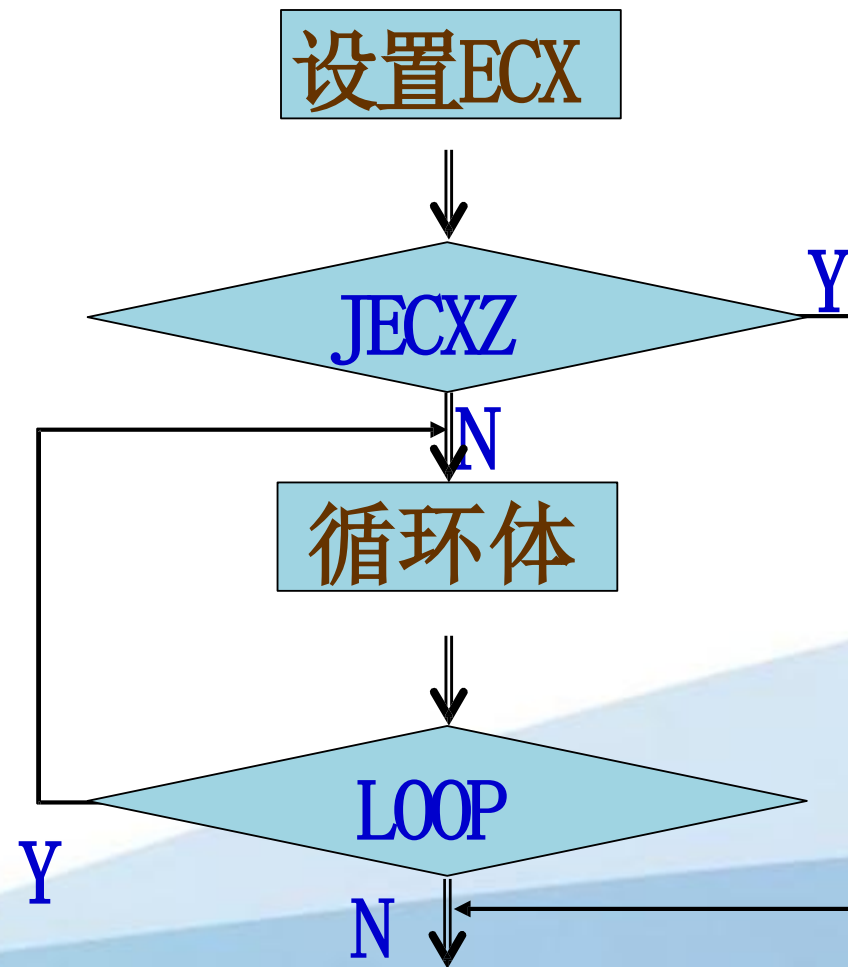
JECXZ label

; ECX=0, 转移到label

; 否则, 顺序执行

CMP ECX, 0

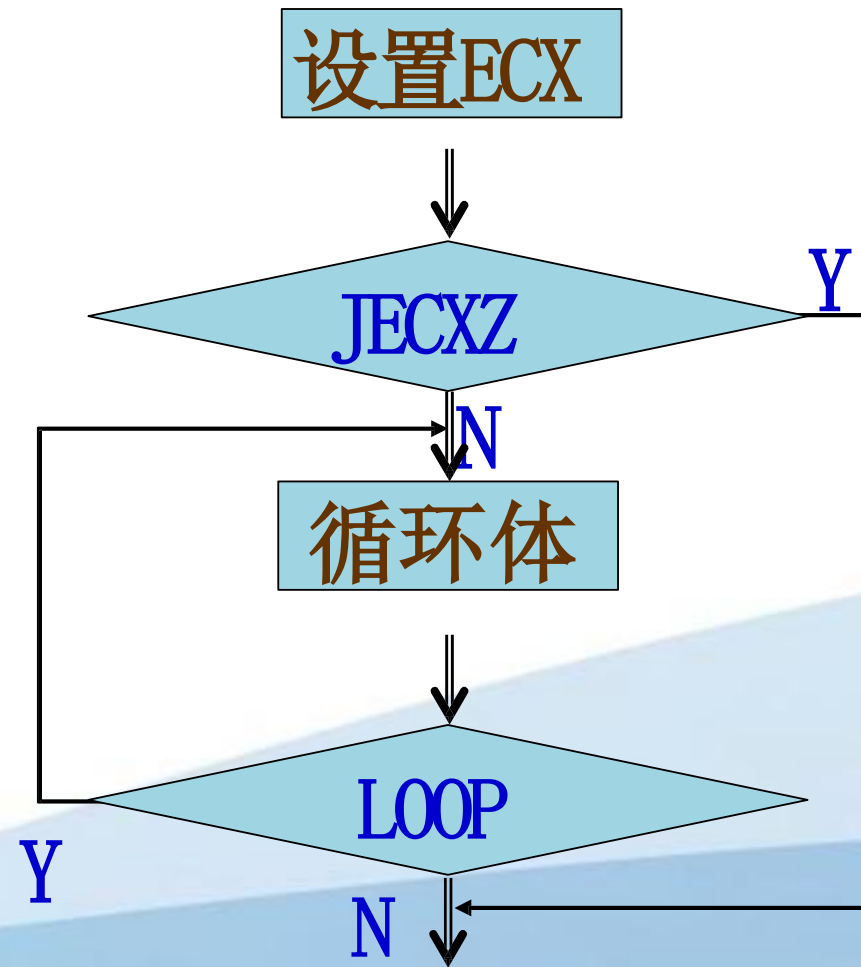
JZ label



JECXZ指令

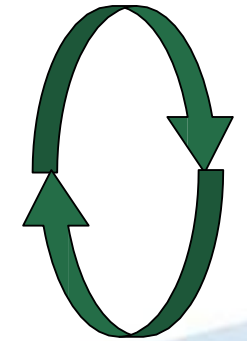
● 典型应用场景

- 防止循环次数为0：比如从外部读取一个“循环次数”到 ECX，但外部可能传入 0（无效次数），此时用 JECXZ跳过循环，避免执行无意义的循环体。
- 安全处理ECX的初始状态：在一些复杂逻辑中，ECX可能被多次修改，用 JECXZ确保“只有当 $ECX \neq 0$ 时，才进入依赖ECX的循环/逻辑”。



本讲总结

- LOOP指令，用于实现减量计数的循环控制
 - 设置ECX等于计数初值
 - ECX通常不应为0（实际表达 2^{32} 次循环）
- JECXZ指令用于跳过ECX=0的情况



汇编语言程序设计

4.3 循环程序结构

2. LOOP循环指令案例 数组求和程序

数组求和

- 数组元素逐个相加（不考虑溢出）
- 元素逐个相加，作为循环体
- 数组元素个数已知，可用LOOP指令控制计数

;数据段

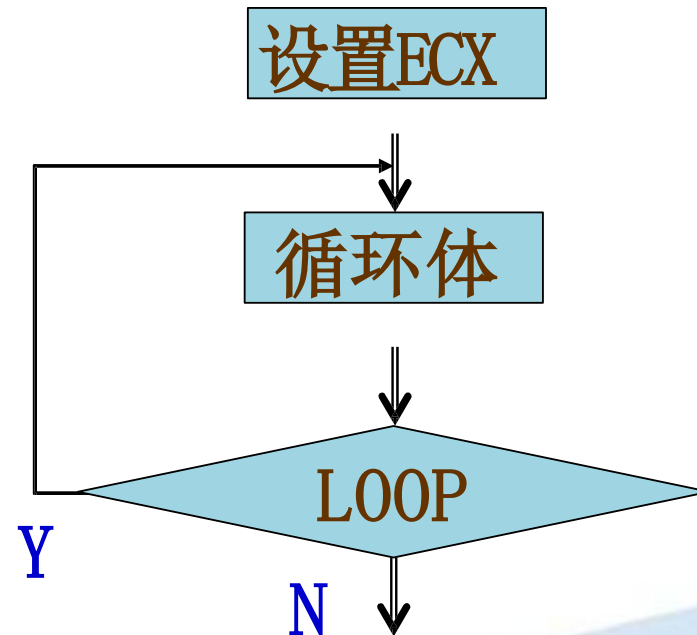
array dword 136, -138, 133, 130, -161 ;数组

sum dword ? ;结果变量

数组求和

● LOOP label

- ;功能1: $ECX = ECX - 1$
- ;功能2: 若 $ECX \neq 0$, 转移到 LABEL
- ;否则, 顺序执行



LOOP指令的应用

思考：label、loop_label如何对应C语言中的循环代码块？

```
mov ecx, num
```

```
label:
```

```
    loop_code
```

```
    LOOP label:
```

```
next code
```

;设置循环的计数初值num

;循环体

;ECX减1，未到0继续循环

;到0循环结束，顺序执行

```
int count = num;  
while count!=0: {  
    循环体代码  
    count=count-1;  
}
```

迁移到数组求和程序

- LOOP 是循环指令，用于实现减量计数的循环控制
- 控制好ECX=求和次数

```
mov ecx, lengthof array    ;ECX=数组元素个数
xor  eax, eax              ;求和初值为0
mov  ebx, offset array     ;指向首个元素
again:
add  eax, [ebx]            ;求和
add  ebx, 4                ;指向下一个数组元素
loop again
mov  sum, eax              ;保存结果
```

循环程序数组 思考-寻址方式

思考：如何在循环程序总有效便捷访问数组数据？

- 寻址存储器内的操作数，使用存储器寻址
- 存储器的直接寻址适用于访问单个变量
 - 但不方便改变地址,故不适合访问数组元素
- 存储器的寄存器间接、相对和变址都使用寄存器
 - 通过修改寄存器内容改变地址， 可方便地访问数组元素

数组求和程序--间接寻址方法

- LOOP 是循环指令，用于实现减量计数的循环控制
- EBX赋值数组首个元素的地址
- 地址增量4，因为 数组元素是32位、占4个字节地址空间

```
mov ecx, lengthof array    ;ECX=数组元素个数
xor  eax, eax               ;求和初值为0
mov  ebx, offset array      ;EBX指向首个元素
again:
add  eax, [ebx]             ;求和
add  ebx, 4                 ;指向下一个数组元素
loop again
mov  sum, eax               ;保存结果
```


数组求和程序--间接寻址方法

- 对应C语言 ebx对应着访问数据的指针

```
int array[] = {136, -138, 133, 130, -161};  
int sum = 0;  
int *p = array;  
int count = 5;  
while (count != 0) {  
    sum = sum + *p;  
    p++;  
    count--;  
}
```

数组求和程序--相对寻址方法

- LOOP 是循环指令，用于实现减量计数的循环控制
- EBX赋值距离数组首个元素的位移量
- 地址增量4，因为数组元素是32位、占4个字节地址空间

```
mov ecx, lengthof array    ;ECX=数组元素个数
xor  eax, eax               ;求和初值为0
mov  ebx, 0                 ;array[EBX]是首个元素的地址
again:
add  eax, array[ebx]        ;求和, 数组元素是4个字节数据
add  ebx, 4                 ;元素个数加1, 指向下一个数组元素
loop again
mov  sum, eax               ;保存结果
```

迁移到数组求和程序--变址寻址方法

- LOOP 是循环指令，用于实现减量计数的循环控制
- EBX赋值距离数组首个元素的位移量
- 地址增量4，因为数组元素是32位、占4个字节地址空间

```
mov ecx, lengthof array    ;ECX=数组元素个数
xor  eax, eax               ;求和初值为0
mov  ebx, 0                 ;array[EBX]是首个元素的地址
again:
add  eax, array[ebx*(type array)] ;求和, 数组元素是4个字节数据
add  ebx, 1                 ;元素个数加1, 指向下一个数组元素
loop again
mov  sum, eax               ;保存结果
```

数组求和程序--相对/变址寻址方法

- 对应C语言：ebx对应着数组的偏移量

```
int array[] = {136, -138, 133, 130, -161};  
int sum = 0;  
int i = 0;  
int count=5  
while (count!=0) {  
    sum = sum + array[i];  
    i++;  
    count=count-1;  
}
```

复习：全部寻址方式总对比

寻址方式	典型语句	关键含义	C场景	记忆点
寄存器寻址	<code>mov ecx, eax</code>	寄存器中的数据传送	中间结果	无
立即数寻址	<code>add eax, 5</code>	常量写在指令中	常量赋值	数直接给出
直接寻址	<code>movecx, count</code>	变量名给出地址	普通变量	名字就能定位
间接寻址	<code>mov edx, [ebx]</code>	按寄存器地址取值	指针*P	寄存器里装地址
相对寻址	<code>mov esi, [ebx+4]</code>	基址+常量偏移	<code>arr[1]</code>	偏移按字节
相对寻址	<code>mov eax, count[esi]</code>	符号地址+寄存器偏移	<code>count[i]</code>	基址+位移
变址寻址	<code>mov eax, [ebx+edx+80h]</code>	基址+变址+位移	<code>buf[32+i]</code>	多一个动态偏移
比例变址	<code>mov eax, [ebx+esi*4-80h]</code>	基址+下标 $\times$ 大小+位移	<code>arr[i - 32]</code>	自动乘元素大小



总结

模块	关键内容
循环三要素	循环初始（设置条件）→ 循环体（重复执行）→ 循环控制（判断继续）
LOOP指令	功能：ECX=ECX-1，若ECX≠0则跳转；等价于 DEC ECX + JNZ label
ECX=0陷阱	初值为0时，-1的补码=FFFFFFFFH=2 ³² -1，导致循环2 ³² 次
JECXZ指令	用于跳过ECX=0的情况，避免上述陷阱

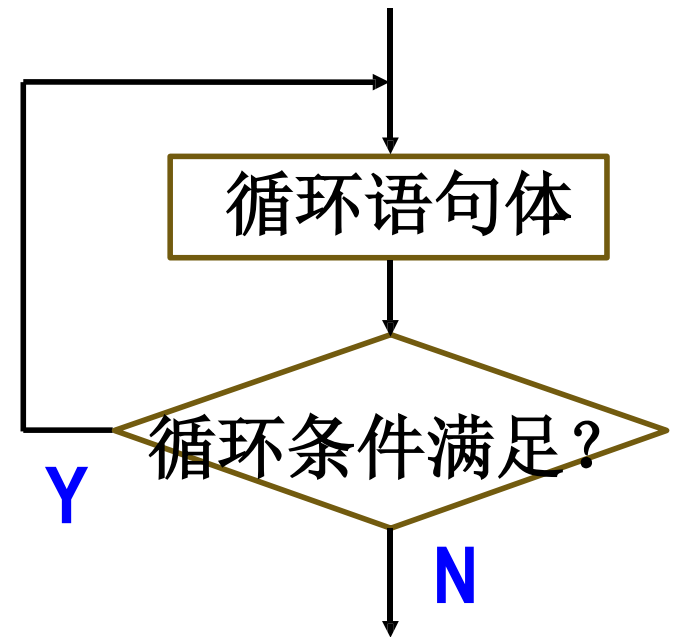
汇编语言程序设计

4.3 循环程序结构

3. 计数控制循环程序

计数控制循环

- 通过次数控制循环
 - 计数可以减量进行，即减到0结束
 - 计数可以增量进行，即达到规定值结束
- 常见是“先循环、后判断”循环结构



计数控制循环--求数组最大值

- 从第1个元素开始逐个比较，保留较大值 元素个数作为循环次数，实现计数控制 循环结束，保留值就是最大值
- 思考：使用LOOP/不使用LOOP？

;数据段

array dword -3, 0, 20, 900, -56

count = lengthof array ;数组的元素个数

max dword ? ;存放最大值

计数控制循环--使用LOOP

- ECX=固定的计数次数

;代码段

mov ecx, count-1 ;元素个数减1是循环次数

mov esi, offset array

mov eax, [esi] ;第一个元素暂存为最大值

again:

add esi, 4

cmp eax, [esi] ;与下一个数据比较

jge next ;循环体包含一个单分支程序，已是较大值，继续

mov eax, [esi]; EAX取得更大的数据

next:

loop again;计数循环

mov max, eax

先循环

后判断

计数控制循环--使用LOOP

- ECX=固定的计数次数，loop指令判断条件

;代码段

mov ecx, count-1 ;元素个数减1是循环次数

mov esi, offset array

mov eax, [esi] ;第一个元素暂存为最大值

again:

add esi, 4

cmp eax, [esi] ;与下一个数据比较

jge next ;循环体包含一个单分支程序，已是较大值，继续

mov eax, [esi]; EAX取得更大的数据

next:

loop again;计数循环

mov max, eax

先循环

后判断

计数控制循环--不使用LOOP

;代码段

```
xor esi,esi
```

; (初始化ESI为0, 用于数组索引)

```
mov eax,array[esi]
```

; EAX保存最大值(将数组第一个元素存入EAX)

again:

;循环开始标签

```
add esi,1
```

; 增量计数(ESI加1, 指向下一个数组元素)

```
cmp eax,array[esi*4]  
esi*4寻址)
```

; 比较EAX和当前数组元素(数组元素是4字节, 所以用

```
jge next
```

; 如果EAX >= 当前元素, 跳转到next(保持EAX为较大值)

```
mov eax,array[esi*4]
```

; 否则, 更新EAX为当前较大的元素

next:

;跳转目标标签

```
cmp esi,count-1
```

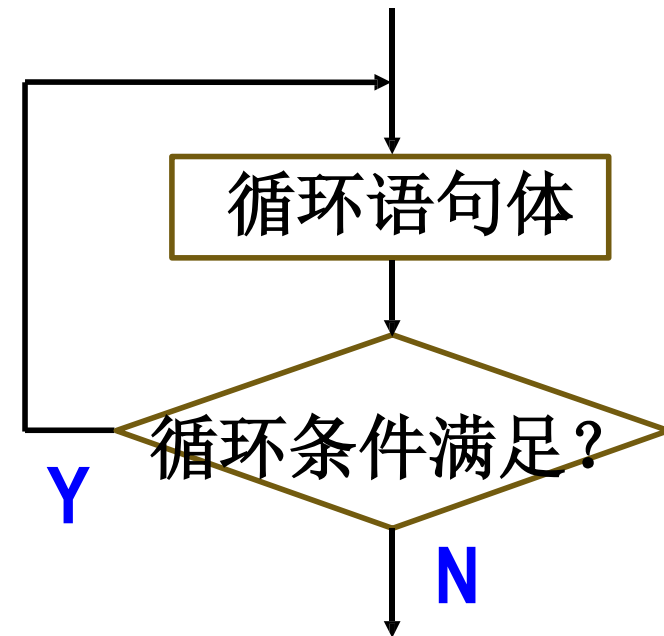
; 比较索引和“元素个数-1”(判断是否遍历完所有元素)

jb again

;如果索引小于count-1, 继续循环(计数循环)

```
mov max,eax
```

; 保存最大值(将最终的最大值存入max变量)



计数控制循环--不使用LOOP

- count=固定的计数次数；通过add esi,1来进行手动的增量

```
;代码段
xor esi,esi                ;(初始化ESI为0, 用于数组索引)
mov eax,array[esi]         ;EAX保存最大值(将数组第一个元素存入EAX)
again:                     ;循环开始标签
add esi,1                  ;增量计数(ESI加1, 指向下一个数组元素)
cmp eax,array[esi*4]       ;比较EAX和当前数组元素(数组元素是4字节, 所以用esi*4寻址)
jge next                  ;如果EAX >= 当前元素, 跳转到next(保持EAX为较大值)
mov eax,array[esi*4]       ;否则, 更新EAX为当前较大的元素
next:                      ;跳转目标标签
cmp esi,count-1            ;比较索引和“元素个数-1”(判断是否遍历完所有元素)
jb again                  ;如果索引小于count-1, 继续循环(计数循环)
mov max,eax                ;保存最大值(将最终的最大值存入max变量)
```

先循环

再判断

计数控制循环--不使用LOOP

- count=固定的计数次数；通过cmp esi,count-1来进行判断

;代码段

```
xor esi,esi           ;(初始化ESI为0, 用于数组索引)
mov eax,array[esi]    ;EAX保存最大值(将数组第一个元素存入EAX)
again:                ;循环开始标签
add esi,1              ;增量计数(ESI加1, 指向下一个数组元素)
cmp eax,array[esi*4]  ;比较EAX和当前数组元素(数组元素是4字节, 所以用esi*4寻址)
jge next              ;如果EAX >= 当前元素, 跳转到next(保持EAX为较大值)
mov eax,array[esi*4]  ;否则, 更新EAX为当前较大的元素
next:                  ;跳转目标标签
cmp esi,count-1       ;比较索引和“元素个数-1”(判断是否遍历完所有元素)
jb again              ;如果索引小于count-1, 继续循环(计数循环)
mov max,eax           ;保存最大值(将最终的最大值存入max变量)
```

先循环

再判断

汇编语言程序设计

4.3 循环程序结构

4. 条件控制循环

条件控制循环

- 根据条件决定是否进行循环
 - 使用比较、测试等指令设置状态标志、产生条件
 - 使用条件转移指令实现循环控制
 - 常需要使用无条件转移指令配合实现循环
- 多见是“先判断、后循环”循环结构
 - 对应高级语言的while语句

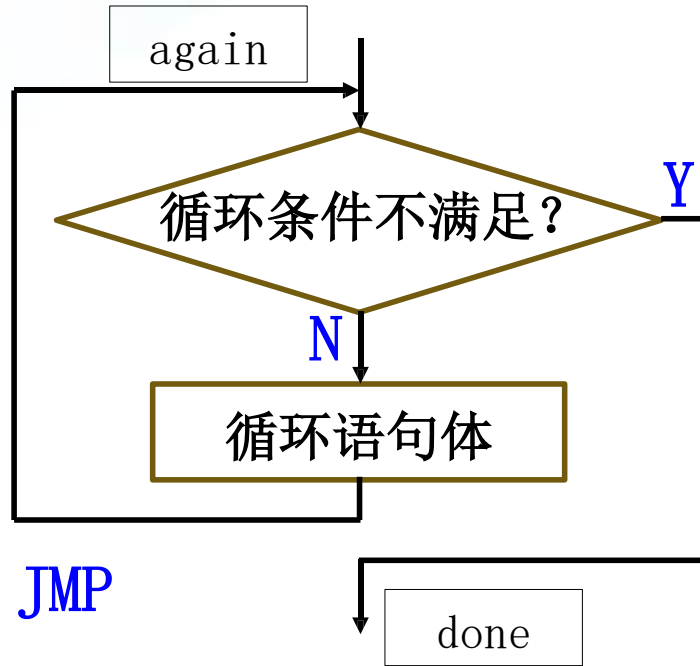
先行判断和先行循环

```
while (条件是否满足) {  
    循环语句体;  
}
```

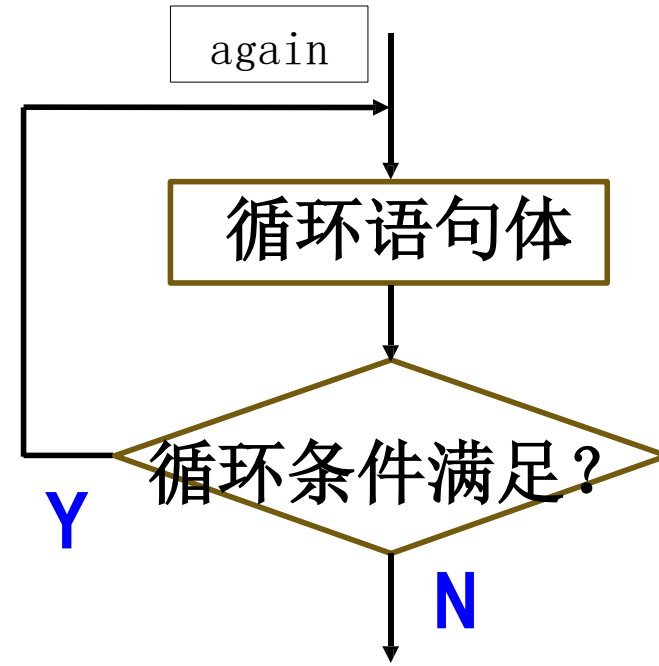
```
do {  
    循环语句体;  
} while (条件);
```

	先判断后循环	先循环后判断
对应结构	while	do...while
判断时机	循环体之前	循环体之后
是否可能一次都不执行	可以	不可以

先行判断和先行循环



C语言:
while (条件是否满足) {
 循环语句体;
}



C语言:
do {
 循环语句体;
} while (条件);

统计以0结尾的字符串个数

;数据段

string byte 'Do you have fun?', 0 ;以0结尾的字符串

;代码段

xor ebx, ebx ;EBX用于记录字符个数;也用于指向字符的指针

字符不是0，个数增量1 字符是0，表示字符串结束
字符是否为0就是判断循环是否结束的条件

统计以0结尾的字符串个数

先行判断

```
include irvine32.inc

.data
    string BYTE 'Do you have fun?',0
    msg     BYTE 'Length = ',0

.code
main PROC
    xor ebx, ebx                ; EBX = 0, 既作下标, 也作字符计数

again:
    cmp string[ebx], 0         ; 先判断当前字符是否为 0
    jz  done                  ; 如果是 0, 结束循环
    inc ebx                    ; 不是 0, 字符个数加 1
    jmp again                  ; 返回继续判断

done:
    mov edx, OFFSET msg
    call WriteString
    mov eax, ebx
    call WriteInt
    call Crlf

    exit
main ENDP
END main
```

先行循环

```
include irvine32.inc

.data
    string BYTE 'Do you have fun?',0
    msg     BYTE 'Length = ',0

.code
main PROC
    mov ebx, -1                ; 先设为 -1, 这样第一次inc后正好检查string[0]

again:
    inc ebx                    ; 先循环一步
    cmp string[ebx], 0         ; 再判断当前字符是否为 0
    jnz again                  ; 如果不是 0, 继续循环

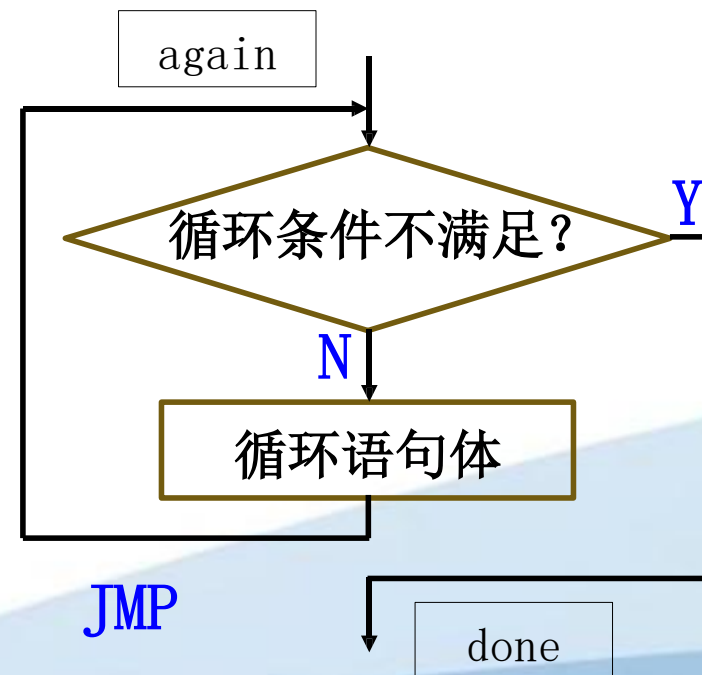
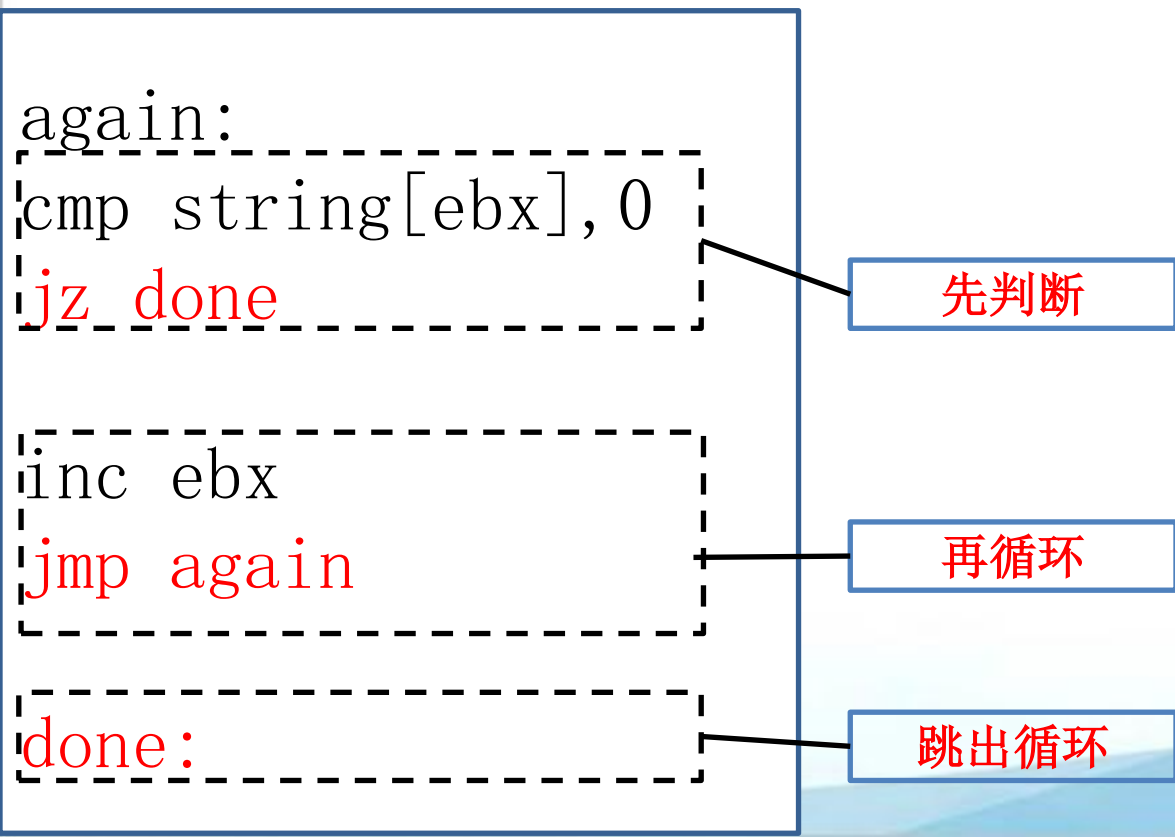
done:
    mov edx, OFFSET msg
    call WriteString
    mov eax, ebx
    call WriteInt
    call Crlf

    exit
main ENDP
END main
```

done标签可以省去

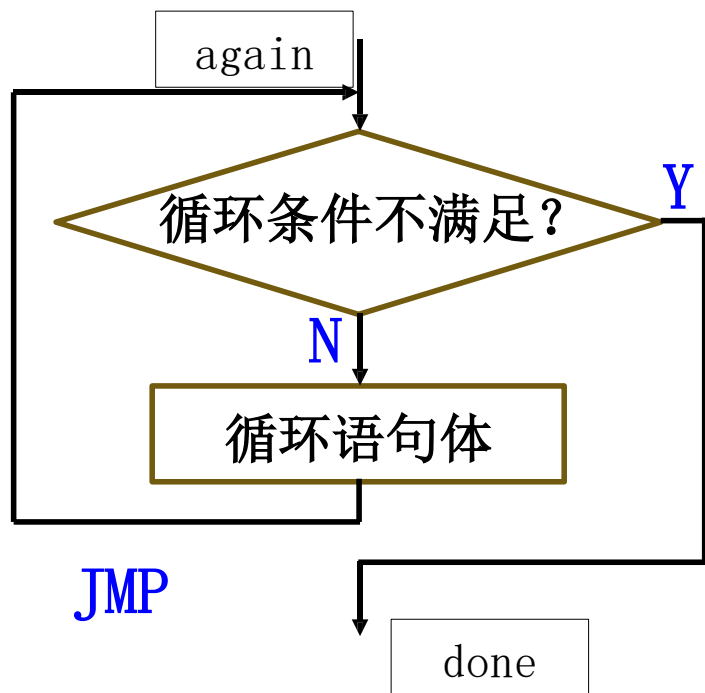
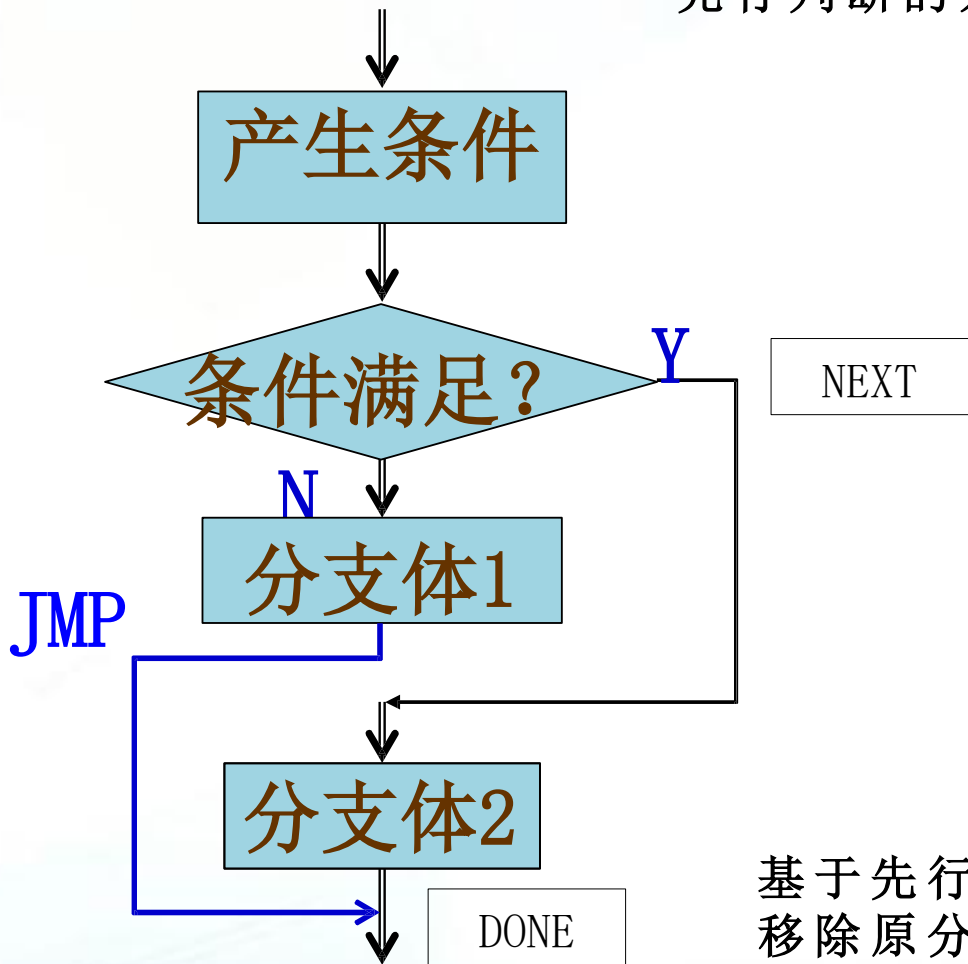
先行判断的条件控制循环

- 顺序执行的分支 → 需要重复执行多次，即循环体
- 另一个分支 → 用于跳出这个循环



先行判断的条件控制循环

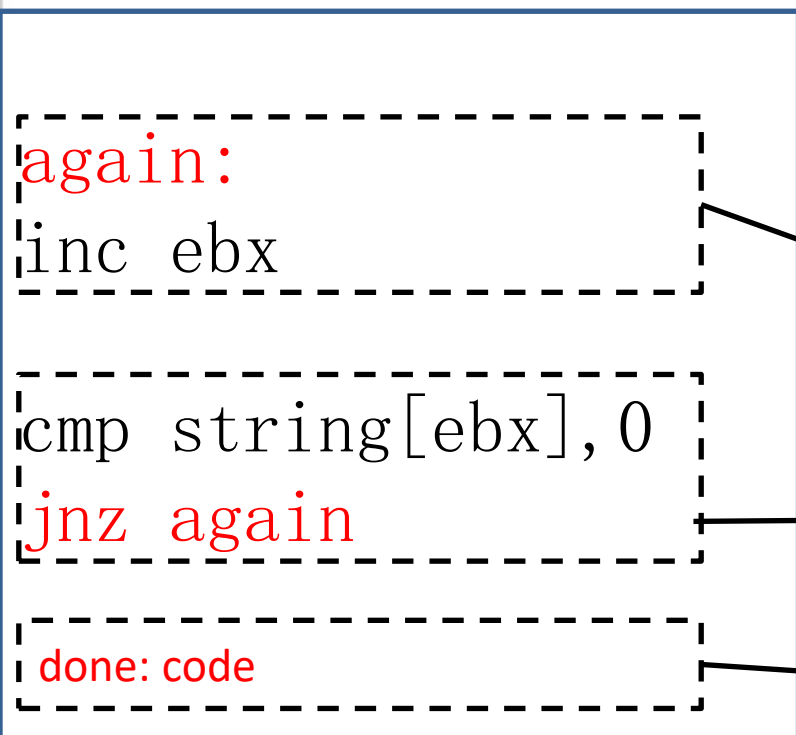
先行判断的条件控制循环类似双分支结构



基于先行判断的条件控制循环与双分支结构类似：
移除原分支体2，将分支体1的JMP目标改至循环条件判断前

先行循环的条件控制循环

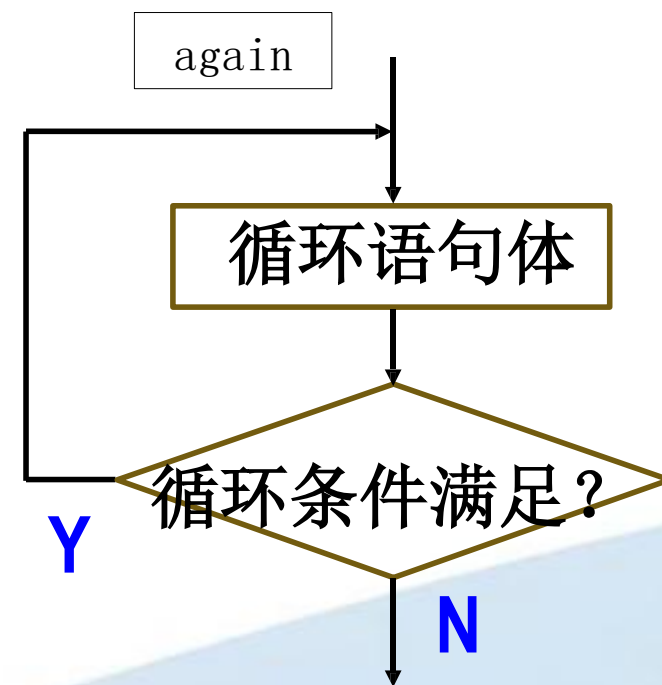
- 分支体就是循环体
- 顺序执行就跳出循环



先循环

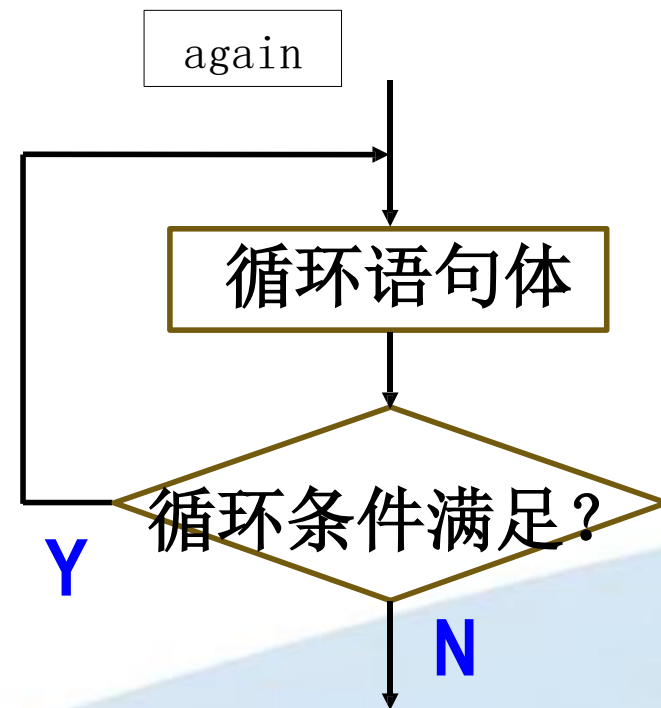
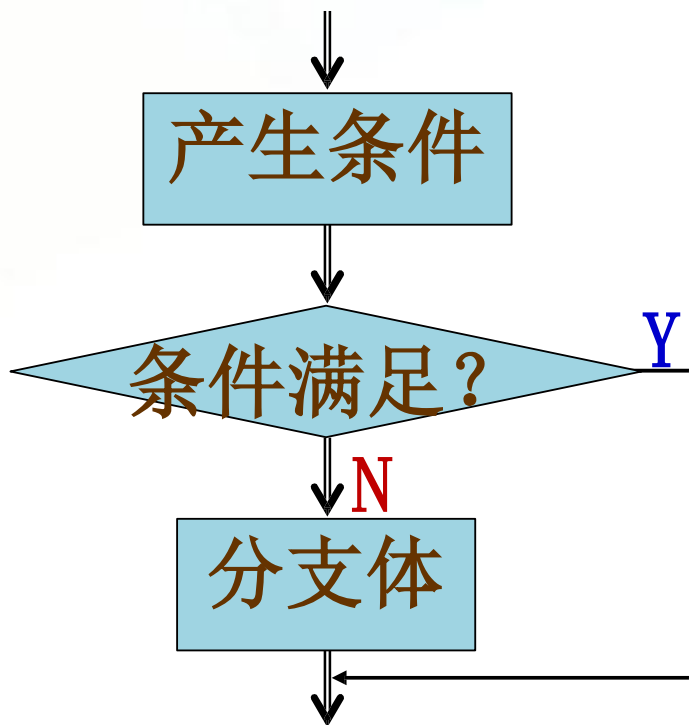
再判断

顺序执行跳出循环,done标签可以省去



条件控制循环

先行循环的条件控制循环类似单分支结构



基于先行循环的条件控制循环与单分支结构类似：JCC的对象是AGAIN